Web Development Foundations

Week 6 - Debugging: "Things Break - And I Can Fix Them"

Day 2: "Debugging Multiple Issues"

1

Reconnect

Monday:

- reproduced one problem
- read one console clue
- compared HTML and JavaScript
- fixed one selector
- verified the result

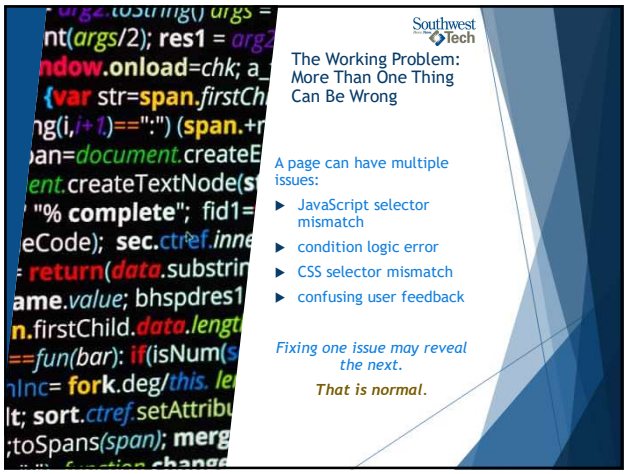
Today:

- we keep that process when more than one issue appears.

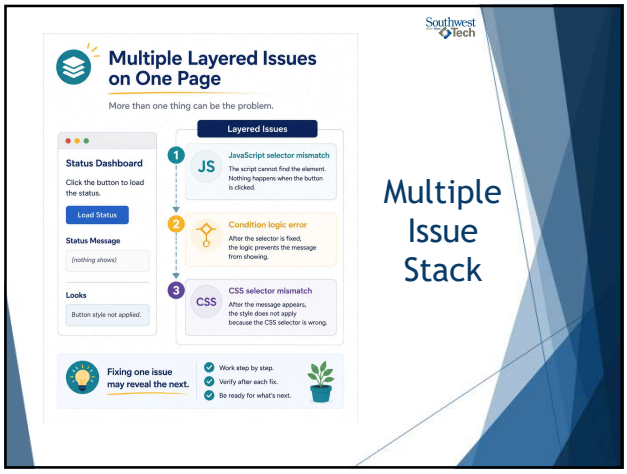
2

One Bug To Issue List

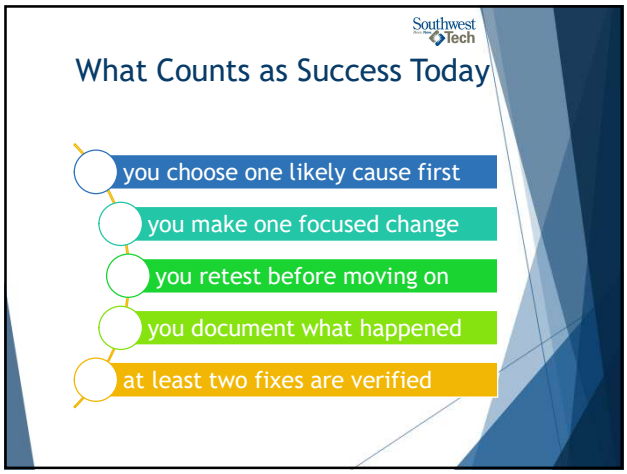
3


The Working Problem: More Than One Thing Can Be Wrong
A page can have multiple issues:
JavaScript selector mismatch
condition logic error
CSS selector mismatch
confusing user feedback
Fixing one issue may reveal the next.
That is normal.

4


Multiple Layered Issues on One Page
More than one thing can be the problem.
Layered Issues
Status Dashboard
Click the button to load the status.
Load Status
Status Message
(nothing shows)
Looks
Button style not applied.
JavaScript selector mismatch
The script cannot find the element. Nothing happens when the button is clicked.
Condition logic error
After the selector is fixed, the logic prevents the message from showing.
CSS selector mismatch
After the message appears, the style does not apply because the CSS selector is wrong.
Fixing one issue may reveal the next.
Work step by step.
Verify after each fix.
Be ready for what's next.
Multiple Issue Stack

5


What Counts as Success Today
you choose one likely cause first
you make one focused change
you retest before moving on
you document what happened
at least two fixes are verified

6

Prioritize The First Cause

Start with the issue that blocks testing.

Good first checks:

- script file linked?
- console error present?
- selector matches HTML?
- event listener connected?

Cosmetic issues can wait until behavior works.

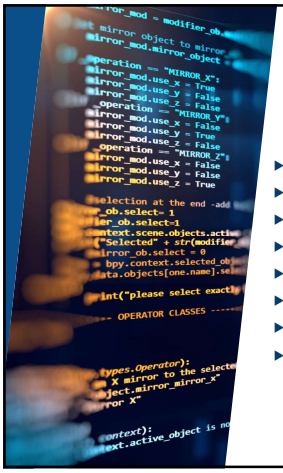
7

Priority Ladder

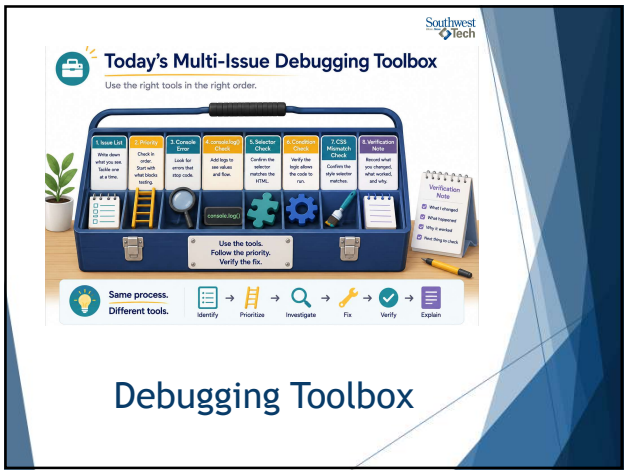
8

What We Will Use Today

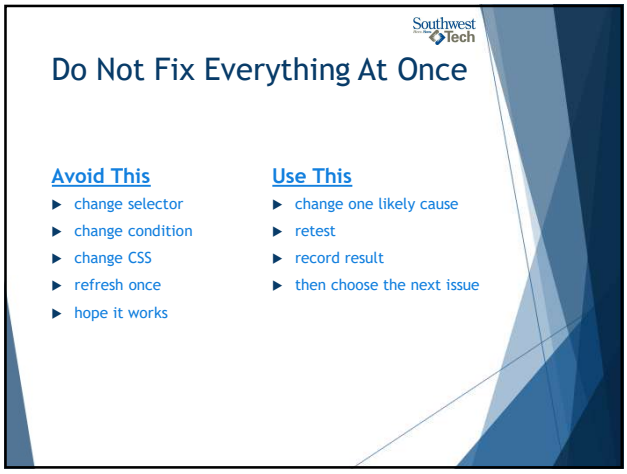
- issue list
- priority
- console error
- `console.log()`
- selector check
- condition check
- CSS mismatch check
- verification note



9


Southwest Tech
Today's Multi-Issue Debugging Toolbox
Use the right tools in the right order.
1. Issue List
2. Priority
3. Console Error
4. console.log() Check
5. Selector Check
6. Condition Check
7. CSS Mismatch Check
8. Verification Note
Use the tools.
Follow the priority.
Verify the fix.
Verification Note
Same process.
Different tools.
Identify
Prioritize
Investigate
Fix
Verify
Explain
Debugging Toolbox

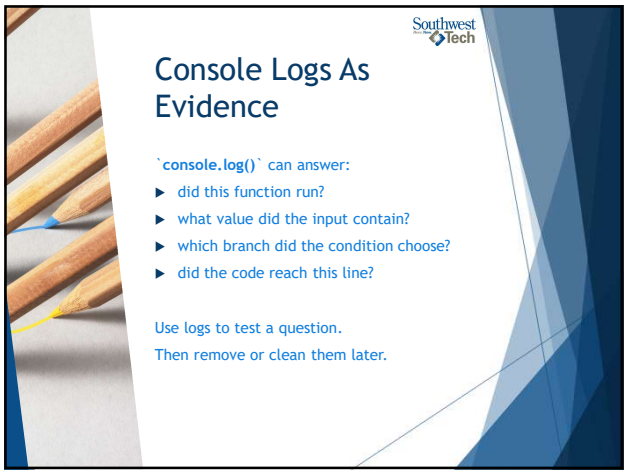
10


Southwest Tech
Do Not Fix Everything At Once
Avoid This
change selector
change condition
change CSS
refresh once
hope it works
Use This
change one likely cause
retest
record result
then choose the next issue

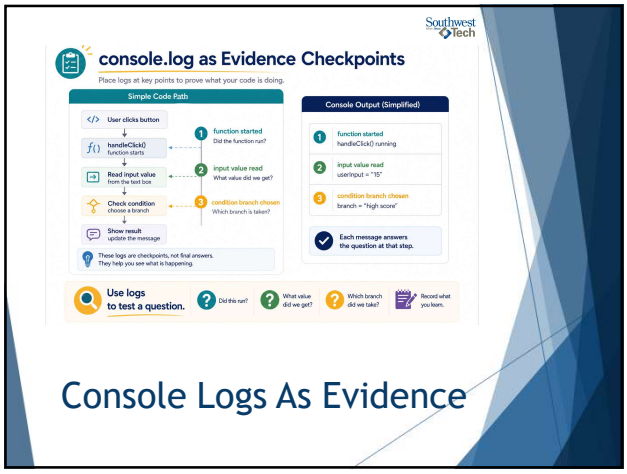
11


Southwest Tech
Debug Smarter, Not Harder
Two ways to handle multiple issues.
Chaotic Approach
Change Everything
change this
and this
maybe this
also this
and this too
Hard to know what fixed it.
New problems can appear.
VS.
Calm Approach
One Fix at a Time
1
change one cause
Pick the next most likely issue and change only that.
2
retest
Run the same test or action.
3
record result
What changed? What stayed the same?
4
choose next issue
Use your results to decide what to tackle next.
Clear results. Steady progress.
Fewer new problems.
Retesting tells you what worked.
Small, targeted changes.
Clear evidence from each step.
Confident fixes, one at a time.
One Fix At A Time

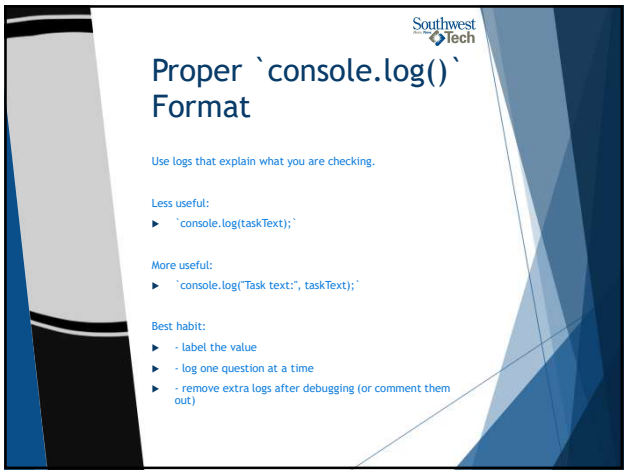
12


Southwest Tech
Console Logs As Evidence
`console.log()` can answer:
did this function run?
what value did the input contain?
which branch did the condition choose?
did the code reach this line?
Use logs to test a question.
Then remove or clean them later.

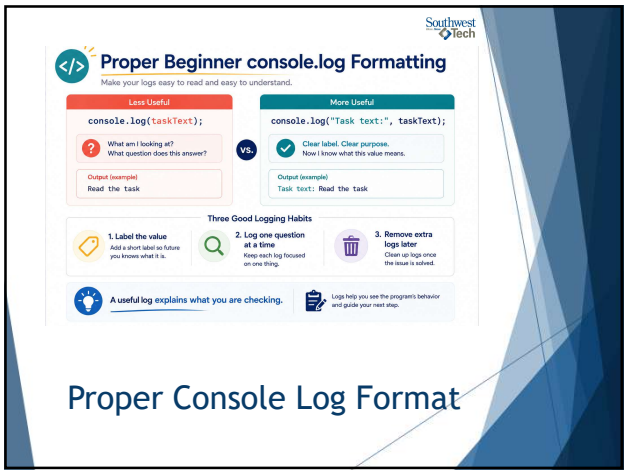
13


Southwest Tech
console.log as Evidence Checkpoints
Place logs at key points to prove what your code is doing.
Simple Code Path
User clicks button
handleClick()
function starts
Read input value
from the text box
Check condition
choose a branch
Show result
update the message
function started
Did the function run?
input value read
What value did we get?
condition branch chosen
Which branch is taken?
These logs are checkpoints, not final answers.
They help you see what is happening.
Console Output (Simplified)
function started
handleClick() running
input value read
userInput = "15"
condition branch chosen
branch = "high score"
Each message answers
the question at that step.
Use logs
to test a question.
Did this run?
What value
did we get?
Which branch
did we take?
Record what
you learn.
Console Logs As Evidence

14


Southwest Tech
Proper `console.log()` Format
Use logs that explain what you are checking.
Less useful:
`console.log(taskText);`
More useful:
`console.log("Task text:", taskText);`
Best habit:
- label the value
- log one question at a time
- remove extra logs after debugging (or comment them out)

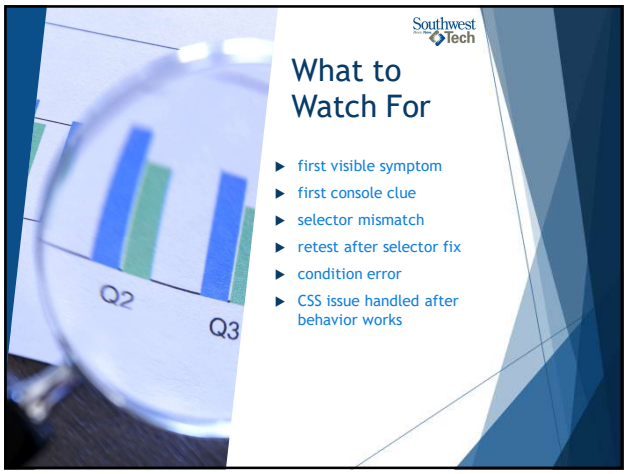
15


Southwest Tech
Proper Beginner console.log Formatting
Make your logs easy to read and easy to understand.
Less Useful
console.log(taskText);
What am I looking at?
What question does this answer?
Output (example)
Read the task
vs.
More Useful
console.log("Task text:", taskText);
Clear label. Clear purpose.
Now I know what this value means.
Output (example)
Task text: Read the task
Three Good Logging Habits
1. Label the value
Add a short label so future you knows what it is.
2. Log one question at a time
Keep each log focused on one thing.
3. Remove extra logs later
Clean up logs once the issue is solved.
A useful log explains what you are checking.
Logs help you see the program's behavior and guide your next step.
Proper Console Log Format

16


Demo
"Multi-Issue Debugging"

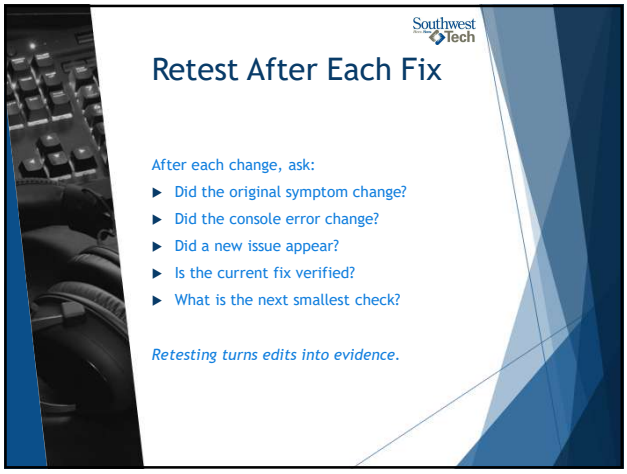
17


Southwest Tech
What to Watch For
first visible symptom
first console clue
selector mismatch
retest after selector fix
condition error
CSS issue handled after behavior works
Q2
Q3

18


Southwest Tech
Multi-Issue Debugging (Recorded Demo)
One page. Three issues. One methodical path.
Task Page
Score Checker
Check Score
Result
Three Issues on This Page
Methodical Path (One Issue at a Time)
fix selector
retest
fix condition
retest
check styling
What You'll See in the Demo
Small, focused fixes
Retest after each fix
Clear progress
Same process. Three issues.
Order matters.
Demo:
Multi-Issue Debugging

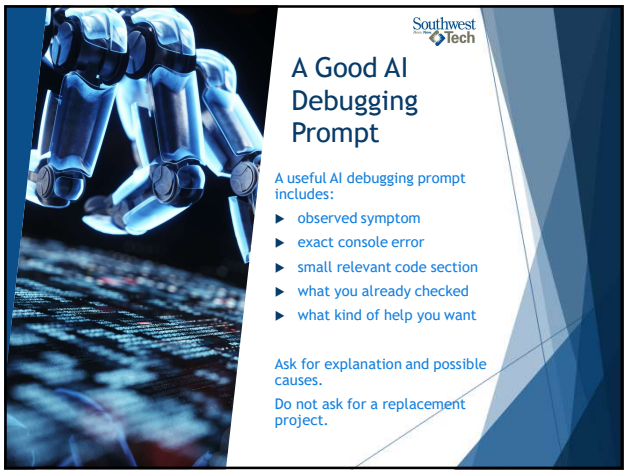
19


Southwest Tech
Retest After Each Fix
After each change, ask:
Did the original symptom change?
Did the console error change?
Did a new issue appear?
Is the current fix verified?
What is the next smallest check?
Retesting turns edits into evidence.

20


Southwest Tech
Retesting After Each Fix
Small change. Retest. Collect evidence.
Fix 1
Make one targeted change.
Retest
Run the same action or test again.
Evidence
Record what happened and what changed.
Fix 2
Make one targeted change.
Retest
Run the same action or test again.
Evidence
Record what happened and what changed.
Fix 3
Make one targeted change.
Retest
Run the same action or test again.
Evidence
Record what happened and what changed.
Retesting turns edits into evidence.
Retest After Each Fix

21



A Good AI Debugging Prompt

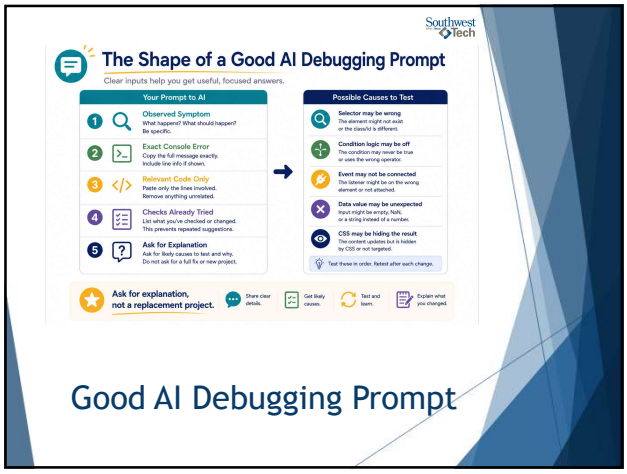
A useful AI debugging prompt includes:

- observed symptom
- exact console error
- small relevant code section
- what you already checked
- what kind of help you want

Ask for explanation and possible causes.

Do not ask for a replacement project.

22



Good AI Debugging Prompt

23

Debugging Report Pattern

Debugging report pattern:

Issue:

- what was wrong?

Evidence:

- how did you identify it?

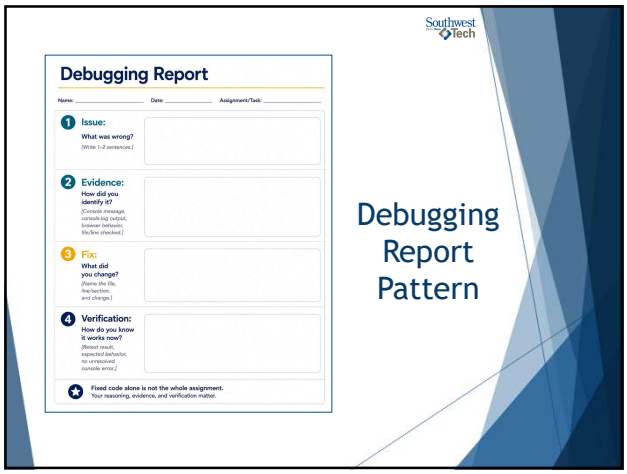
Fix:

- what did you change?

Verification:

- how do you know it works now?

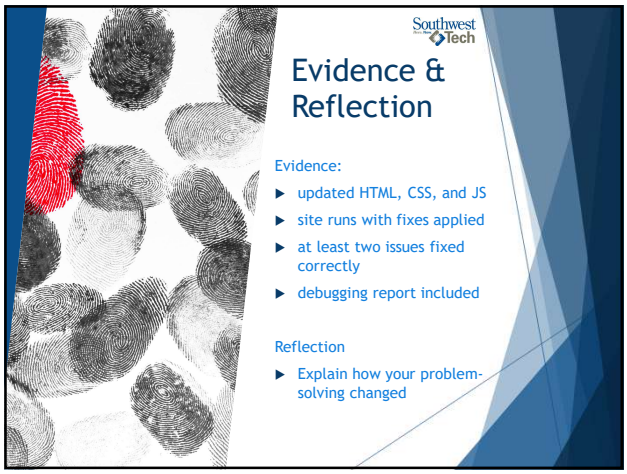
24


Debugging Report
Name: Date: Assignment/Task:
1 Issue:
What was wrong?
[Write 1–2 sentences.]
2 Evidence:
How did you identify it?
[Console message, console.log output, browser behavior, file/line checked.]
3 Fix:
What did you change?
[Name the file, line/section, and change.]
4 Verification:
How do you know it works now?
[Retest result, expected behavior, no unresolved console error.]
Fixed code alone is not the whole assignment.
Your reasoning, evidence, and verification matter.
Debugging Report Pattern

25


Lab Refinement
Verified fixes
Your goal:
fix at least 2 confirmed issues completely
use console output or inspection as evidence
verify the behavior after each fix
document issue, evidence, fix, and verification

26


Evidence & Reflection
Evidence:
updated HTML, CSS, and JS
site runs with fixes applied
at least two issues fixed correctly
debugging report included
Reflection
Explain how your problem-solving changed

27

How To Read Next Week's Material

Required:

- read functions as a way to organize behavior
- notice scope, dot notation, and document as a built-in object

Reference:

- objects and built-in methods are there when you need them
- do not memorize every method

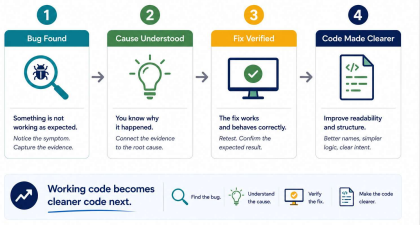
Before next time:

- Bring one piece of interaction code that could be made clearer.

28

From Debugging to Cleaner Behavior

A clear path from problem to better code.



Cleaner Code Bridge

29

Closing Success Check

This week:

- problems became evidence
- console messages became clues
- fixes were verified
- reports explained the process

Next:

- working code becomes cleaner code.

30

31